TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO

Báo cáo Tuần

Môn: Lập trình Đa nền tảng

ĐA LUỒNG VỚI ISOLATES

Tối ưu tác vụ nặng không nghẽn luồng giao diện (Non-blocking UI)

SVTH: Trần Đăng Duy
Phan Văn Trà

Đà Nẵng, 2026

Nội dung báo cáo

Cơ sở lý thuyết

1. Đặt vấn đề, mục tiêu, parse JSON
2. Single-thread và Event Loop
3. Bộ nhớ độc lập của Isolate
4. SendPort, ReceivePort và compute

Ứng dụng FPS Isolate Benchmark

5. Thiết kế và mã nguồn quan trọng
6. Cách đo FPS và độ trễ callback
7. Giao diện, biểu đồ và kết quả
8. Hạn chế, kết luận và tài liệu

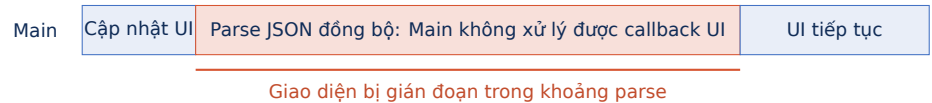
Trọng tâm: cùng một bài toán parse JSON, so sánh ba cách thực thi trên Flutter Native.

Phân công công việc

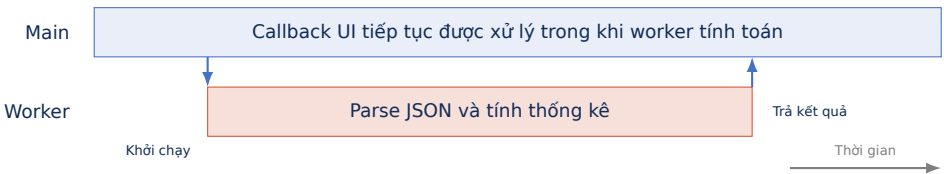
STT	Họ và tên	Nội dung công việc	Tỷ lệ đóng góp
1	Trần Đăng Duy	• Xây dựng giao diện Flutter để chọn chế độ chạy và file JSON. • Hiển thị tiến trình, thời gian parse và tổng thời gian thực thi. • Đo độ mượt UI, frame bị trễ và FPS hoặc các chỉ số frame phù hợp. • Hiển thị kết quả so sánh bằng bảng hoặc biểu đồ. • Viết hướng dẫn chạy demo, chụp kết quả và chuẩn bị slide thuyết trình.	50%
2	Phan Văn Trà	• Xây dựng hàm đọc file JSON và `jsonDecode()`. • Triển khai ba chế độ: Main Thread, `compute()` và `Isolate.spawn()`. • Với `Isolate.spawn()`, xử lý giao tiếp bằng `SendPort` và `ReceivePort`. • Đảm bảo cả ba chế độ thực hiện cùng một tác vụ trên cùng dữ liệu. • Viết unit test kiểm tra kết quả parse, lỗi dữ liệu và tính nhất quán.	50%

1. Đặt vấn đề: Vì sao giao diện bị nghẽn?

A. Parse đồng bộ trên Main Isolate



B. Parse trên Worker Isolate (Native)



Tác vụ CPU dài làm callback UI đến muộn. `async/await` không tự chuyển phép tính sang worker. Sơ đồ minh họa thứ tự, không theo tỉ lệ thời gian và không bảo đảm FPS cố định.

2. Mục tiêu của đề tài

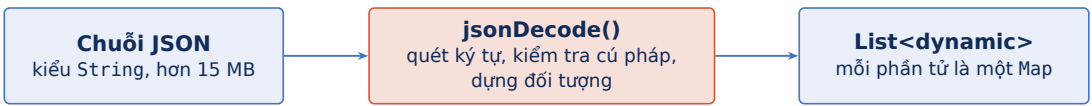
- So sánh xử lý CPU trên Main Isolate với xử lý ở worker isolate.
- Giải thích bộ nhớ độc lập và giao tiếp bằng thông điệp.
- Hiện thực ba chế độ: Main Thread, `compute()`, `Isolate.spawn` kết hợp Ports.
- Dùng JSON lớn hơn 15 MB, hoạt ảnh và biểu đồ FPS để quan sát mức độ nghẽn UI.

Tiêu chí chính: UI tiếp tục cập nhật khi tính toán. Thời gian xử lý là một chỉ số riêng, cần đo độc lập với độ mượt.

3. Parse JSON là gì?

JSON (JavaScript Object Notation) là định dạng văn bản nhẹ để lưu và trao đổi dữ liệu: cặp khóa–giá trị, mảng, chuỗi, số, `true`/`false`, `null`.

Parse JSON là đọc chuỗi JSON, kiểm tra cú pháp rồi chuyển thành đối tượng trong bộ nhớ (`List`, `Map`, ...) để chương trình truy cập. Trong Dart, việc này do `jsonDecode()` đảm nhận.



Dữ liệu JSON (văn bản)

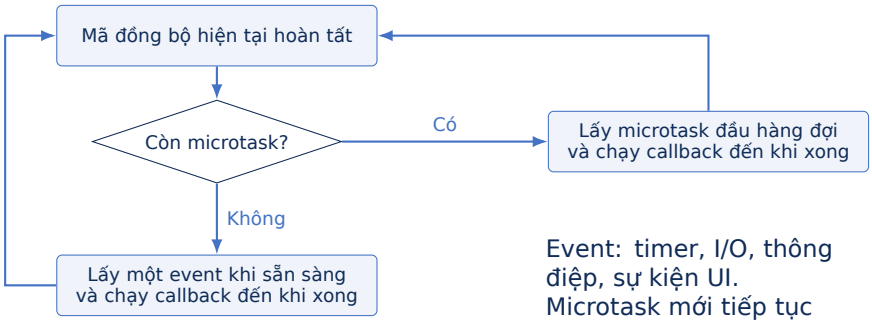
```
[{"name": "An", "score": 8.5,
 "address": "Đà Nẵng",
 "tags": ["flutter", "dart"]}, …]
```

Sau khi parse (Dart)

```
users[0]['name']  // "An"
users[0]['score'] // 8.5
users[0]['tags']  // ["flutter", "dart"]
```

Vì sao tốn CPU? Thời gian parse tăng theo kích thước dữ liệu (khoảng 120.000 đối tượng) và phải tạo rất nhiều đối tượng trong bộ nhớ. `jsonDecode` là hàm đồng bộ, nên gọi trên Main Isolate thì UI phải chờ.

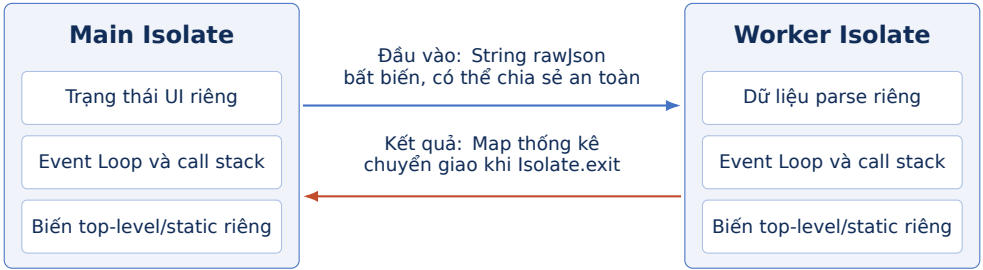
4. Single-thread, Event Loop và Multi-isolate



Event: timer, I/O, thông điệp, sự kiện UI. Microtask mới tiếp tục được ưu tiên trước event kế tiếp.

- Mỗi isolate chạy một callback Dart tại một thời điểm và có Event Loop riêng.
- Callback CPU dài hoặc microtask liên tục có thể trì hoãn event UI. Worker isolate có thể chạy đồng thời với Main trên Native.

5. Bộ nhớ độc lập của Isolate

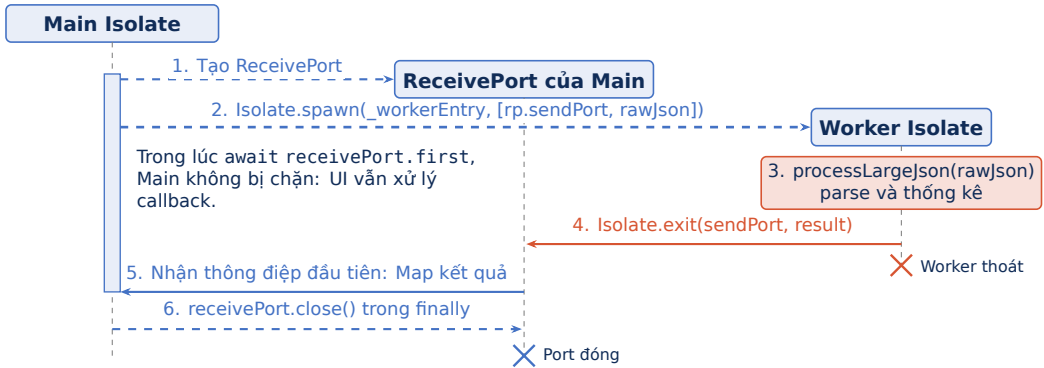


Không chia sẻ quyền sửa cùng đối tượng mutable

- Gửi đối tượng mutable bằng SendPort thường cần sao chép. Với dữ liệu bất biến hoặc chuyển giao khi thoát, cơ chế có thể khác.
- Cô lập trạng thái tránh data race trên đối tượng Dart dùng chung. Thứ tự thông điệp vẫn có thể gây race condition.

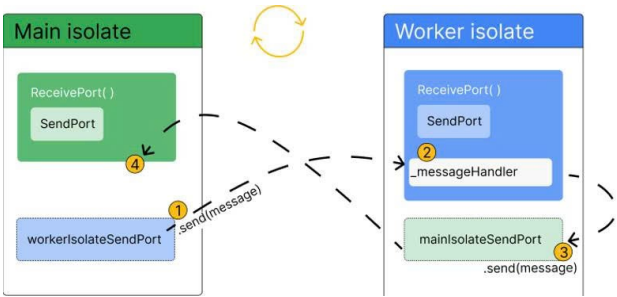
Sơ đồ thể hiện trạng thái logic của isolate, không mô tả cách VM cấp phát heap vật lý.

6. SendPort và ReceivePort



SendPort là địa chỉ gửi đến **ReceivePort**. Worker gửi kết quả qua `Isolate.exit`; nếu worker lỗi, `onError` gửi một List lỗi vào cùng port. Đây là luồng chạy một lần, chưa có handshake hai chiều.

7. Giao tiếp hai chiều giữa các Isolate



1. Main tạo ReceivePort và đưa SendPort của nó cho worker.
2. Worker tạo ReceivePort riêng, gửi SendPort của mình về Main.
3. Hai bên gửi dữ liệu và kết quả qua hai kênh thông điệp.

Đây là mô hình handshake hai chiều. Project hiện dùng worker một lần: nhận JSON lúc spawn, gửi kết quả rồi thoát.

8. Main Thread và hàm compute trong project

```
ProcessResult runOnMain(String rawJson) {
  return processLargeJson(rawJson);
}
Future<ProcessResult> runWithCompute(String rawJson) {
  return compute(processLargeJson, rawJson);
}
```

- `runOnMain`: gọi đồng bộ trên Main Isolate, UI phải chờ.
- `runWithCompute`: trên Native, worker thực hiện cùng hàm và trả kết quả qua Future. Trên Web, callback chạy ở Event Loop hiện tại.
- Callback, dữ liệu và kết quả phải gửi được qua isolate. Hàm top-level là lựa chọn gọn, không phải điều kiện bắt buộc.

9. Isolate + Ports: mã nguồn thực tế

Main tạo worker và chờ kết quả

```
final receivePort = ReceivePort();

try {
  await Isolate.spawn<List<dynamic>>(_worke
  ↪  rEntry, [
    receivePort.sendPort,
    rawJson,
  ], onError: receivePort.sendPort);

  final message = await receivePort.first;
```

Worker xử lý rồi kết thúc

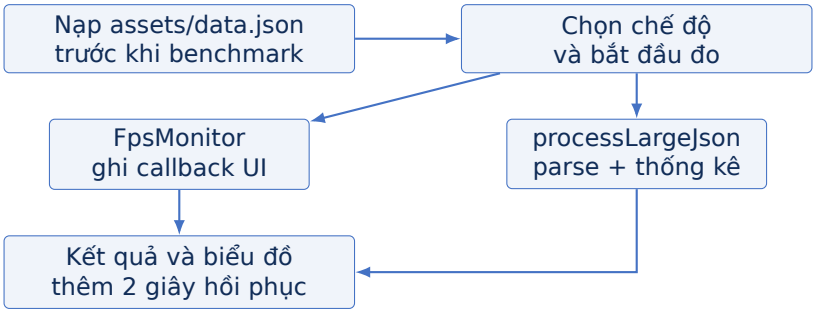
```
void _workerEntry(List<dynamic>
↪  arguments) {
  final sendPort = arguments[0] as
  ↪  SendPort;
  final rawJson = arguments[1] as String;

  final result =
  ↪  processLargeJson(rawJson);
  Isolate.exit(sendPort, result);
}
```

`receivePort.first` nhận thông điệp đầu tiên. Mã đầy đủ phân biệt Map kết quả với List lỗi và đóng port trong `finally`.

`Isolate.exit` có thể chuyển quyền sở hữu kết quả khi worker kết thúc. compute/Isolate.run cũng dùng cơ chế này, nên không thể giải thích chênh lệch FPS chỉ bằng zero-copy.

10. Thiết kế ứng dụng FPS Isolate Benchmark



Chuỗi JSON được nạp trước khi đo. Ba chế độ gọi cùng hàm parse và tính thống kê. FpsMonitor theo dõi callback UI trong lúc xử lý và giai đoạn hồi phục.

11. Tác vụ nặng: parse JSON và tính thống kê

Hàm xử lý đầy đủ

```
ProcessResult processLargeJson(String rawJson) {
  final users = jsonDecode(rawJson) as List<dynamic>;
  double totalScore = 0;
  int daNangCount = 0;
  int tagCount = 0;
  for (final element in users) {
    final user = element as Map<String, dynamic>;
    totalScore += (user['score'] as num).toDouble();
    final address = user['address'] as Map<String, dynamic>;
    if (address['city'] == 'Da Nang') {
      daNangCount++;
    }
    final tags = user['tags'] as List<dynamic>;
    tagCount += tags.length;
  }
  return {
    'userCount': users.length,
    'averageScore': users.isEmpty
        ? 0 : totalScore / users.length,
    'daNangCount': daNangCount,
    'tagCount': tagCount,
  };
}
```

Phần tính toán

1. Giải mã JSON
Tạo danh sách người dùng từ chuỗi dữ liệu.

2. Duyệt từng người
Cộng điểm, đếm địa chỉ Đà Nẵng và tổng số tag.

3. Trả kết quả
Chỉ gửi Map thống kê nhỏ về Main.

Lưu ý: Dữ liệu cần đúng cấu trúc score, address và tags.

12. Cách ứng dụng đo FPS

1. **Đếm lần cập nhật hoạt ảnh:** mỗi khi AnimationController gọi `onFrame()`, bộ đo ghi nhận một callback và thời điểm xuất hiện.
2. **Tính FPS:** lấy số callback chia cho thời gian đo. Ví dụ: **15 callback trong 0,25 giây tương ứng 60 FPS**.
3. **Xét riêng lúc xử lý JSON:** bắt đầu đo trước khi gọi hàm xử lý, chốt giai đoạn này ngay khi nhận kết quả.

Khi Main Isolate bị chặn, callback phải chờ. Bộ đo tổng hợp dữ liệu sau đó để thể hiện đoạn FPS giảm trên biểu đồ.

FPS ở đây ước lượng nhịp callback hoạt ảnh, chưa đo trực tiếp số khung hình màn hình hiển thị.

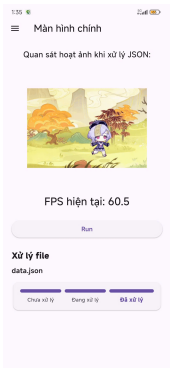
13. Ý nghĩa các chỉ số đánh giá độ mượt

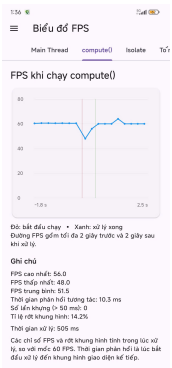
Chỉ số	Ứng dụng đo gì?	Cách hiểu
Độ trễ callback	Thời gian từ lúc bắt đầu benchmark đến callback khung hình kế tiếp.	Càng lớn, UI càng lâu mới có cơ hội cập nhật.
Số lần khựng	Đếm các khoảng cách giữa callback lớn hơn **50 ms**.	Cho biết số khoảng gián đoạn dài của nhịp cập nhật UI.
Tỉ lệ khung hình thiếu	So sánh số callback thực tế với số kỳ vọng ở mốc **60 FPS**.	Tỉ lệ càng cao, số lần cập nhật bị thiếu càng nhiều.

Main bị chiếm dụng sẽ làm callback đến muộn. compute và Isolate chuyển phép tính sang worker, giúp Main tiếp tục cập nhật UI.

Độ trễ callback chưa phải thời gian phản hồi thao tác chạm. Tỉ lệ khung hình thiếu là ước lượng theo mốc 60 FPS.

14. Giao diện và cách chạy demo





1. Chọn Run, lần lượt chạy Main Thread, compute và Isolate + Ports.
2. Quan sát GIF và khả năng UI tiếp tục cập nhật.
3. Mở Biểu đồ FPS để xem từng chế độ và Tổng quan.

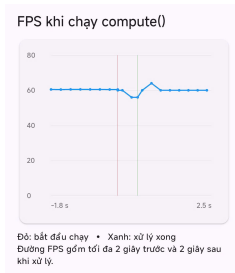
GIF dùng để quan sát. FPS được tính từ callback AnimationController riêng.

15. Đồ thị FPS của ba cách chạy



Main Thread

FPS về 0 trong lúc parse. Main Isolate không chạy được callback UI.



compute()

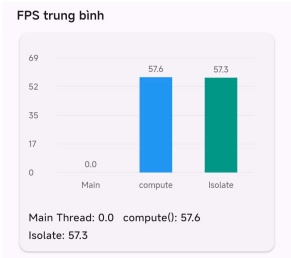
FPS giảm nhẹ nhưng hoạt ảnh vẫn tiếp tục trong lúc worker xử lý.



Isolate + Ports

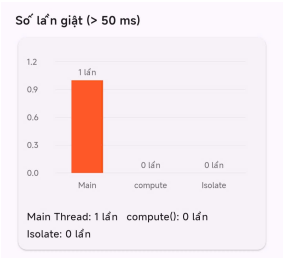
FPS tiếp tục cập nhật. Main nhận kết quả khi worker hoàn thành.

16. Biểu đồ cột so sánh hiệu năng



FPS trung bình

Main: 0,0. compute: 57,6. Isolate: 57,3. Hai chế độ nền duy trì nhịp UI gần 60 FPS.



Số lần khựng

Main có 1 khoảng khựng lớn hơn 50 ms. compute và Isolate không ghi nhận khoảng khựng vượt ngưỡng.



Khung hình thiếu

Ước lượng theo mốc 60 FPS: Main 100%, compute 4,0%, Isolate 4,5%.

17. Bảng phân tích ba cách chạy

Tiêu chí	Main Thread	compute()	Isolate + Ports
Nơi xử lý CPU	Main Isolate	Worker do API quản lý	Worker tạo bằng spawn
Giao tiếp	Gọi hàm đồng bộ	Đối số vào, kết quả qua Future	SendPort, ReceivePort và exit
Ảnh hưởng UI	Nghẽn khi parse	UI tiếp tục cập nhật	UI tiếp tục cập nhật
FPS trung bình	0,0	57,6	57,3
Khựng > 50 ms	1	0	0
Khung hình thiếu	100%	4,0%	4,5%
Mức kiểm soát	Đơn giản	Gọn cho tác vụ một lần	Tự quản lý port và vòng đời

Cả compute và Ports đều tránh giữ Main Isolate trong lúc parse. Chênh lệch nhỏ của một lượt chạy chưa đủ xác lập cách nào luôn tốt hơn.

18. Kết luận và hướng phát triển

Kết luận

- Parse JSON đồng bộ trên Main Isolate làm trì hoãn công việc UI.
- Trên Native, compute và Isolate + Ports cho phép xử lý CPU đồng thời với UI. Lợi ích chính là khả năng phản hồi, không bảo đảm tác vụ luôn nhanh hơn.
- Isolate cô lập trạng thái mutable và trao đổi bằng thông điệp. Thiết kế thứ tự xử lý và tài nguyên ngoài vẫn cần cẩn thận.

Hướng phát triển

- Cho phép chọn tệp JSON và thử nhiều kích thước dữ liệu.
- Đo nhiều lượt trên cùng thiết bị, tổng hợp FPS và thời gian xử lý.
- So sánh trên nhiều thiết bị và màn hình có tần số quét khác nhau.
- Tái sử dụng worker và bổ sung demo lọc ảnh độ phân giải cao.

19. Tài liệu tham khảo

[1] Dart. Concurrency in Dart.
https://dart.dev/language/concurrency

[2] Flutter. Concurrency and isolates.
https://docs.flutter.dev/perf/isolates

[3] Flutter API. compute function.
https://api.flutter.dev/flutter/foundation/compute.html

[4] Dart API. SendPort.send.
https://api.dart.dev/dart-isolate/SendPort/send.html

[5] Flutter. UI performance profiling.
https://docs.flutter.dev/perf/ui-performance

[6] Dart. Announcing Dart 2.15: isolate groups, Isolate.exit.
https://dart.dev/blog/announcing-dart-2-15

CẢM ƠN MỌI NGƯỜI
ĐÃ LẮNG NGHE!

Nhóm xin tiếp nhận câu hỏi và góp ý.